



Department of CSE (Artificial Intelligence & Machine Learning)

II Year II Semester

MACHINE LEARNING LAB (R23AM22L4)

Roll Number: _____

Name of the Student: _____

Academic Year: 2024-2025



Narasaraopeta Engineering College

Kotappakonda Road, Yellamanda (Post), Narasaraopet – 522601, Guntur District, AP

Approved by AICTE, New Delhi & Permanently affiliated to JNTUK, Kakinada, Code: 47,

Accredited by NBA & NAAC, RTA Approved Pollution test Centre, ISO 9001: 2008 Certified
Institution. Phone: 08647-239905, Website: www.nrtec.in.



Approved by AICTE, Permanently affiliated to JNTUK, Accredited by NBA & NAAC with 'A+' Grade,
Kotappakonda Road, Yellamanda (V), Narasaraopeta, Andhra Pradesh – 522601, www.nrtec.in

DEPARTMENT OF
CSE (Artificial Intelligence & Machine Learning)

CERTIFICATE

ACADEMIC YEAR: 2024-25

This is to certify that the bonafide record work done by _____

_____ bearing **Roll Number** _____

of **II B.Tech. II Semester** in the **Machine Learning Laboratory** is satisfactorily
completed.

Staff In-Charge

Head of the Department

Submitted for the practical examination held on _____

External Examiner

INDEX

S.No	Name of the Experiment	Page No	Date	Remarks
1	Mean, Median, Mode Measure of Dispersion: Variance, Standard Deviation	1-3	29-01-2025	
2	Pre-processing techniques for a given dataset. a. Attribute selection b. Handling Missing Values c. Discretization	4-6	05-02-2025	
3	Apply KNN algorithm for classification and regression	7-8	19-02-2025	
4	Demonstrate decision tree algorithm	9-11	05-03-2025	
5	Demonstrate decision tree algorithm for a regression problem	12	09-03-2025	
6	Apply Random Forest algorithm for classification and regression	13	18-03-2025	
7	Demonstrate Naïve Bayes Classification algorithm.	14-15	19-03-2025	
8	Apply Support Vector algorithm for classification	16-18	26-03-2025	
9	Demonstrate simple linear regression algorithm for a regression problem	19-20	01-04-2025	
10	Apply Logistic regression algorithm for a classification problem	21-23	02-04-2025	
11	Demonstrate Multi-layer Perceptron algorithm for a classification problem	24-26	09-04-2025	
12	Implement the K-means algorithm	27-29	16-04-2025	
13	Demonstrate the use of Fuzzy C-Means Clustering	30-31	23-04-2025	
14	Demonstrate the use of Expectation Maximization based clustering algorithm	32-33	30-04-2025	

Week1 (Central Tendency & Measure of Dispersion)

Aim: Find the mean age of employees.

CODE CELL

```
ages = [25, 30, 35, 40, 28, 32, 36, 31, 27, 29]
print(sum(ages) / len(ages))
```

Output:

31.3

Aim: Calculate the median of house prices.

CODE CELL

```
prices = [100000, 150000, 120000, 180000, 200000, 170000, 140000]
prices.sort()
n = len(prices)
median = prices[n//2] if n % 2 != 0 else (prices[n//2 - 1] + prices[n//2]) / 2
print(median)
```

Output:

150000

Aim: Determine the mode of sales data for various products.

CODE CELL

```
from collections import Counter

sales = ["A", "B", "A", "C", "A", "B", "C", "C", "B", "A"]
mode = Counter(sales).most_common(1)[0][0]
print(mode)
```

Output:

A

Aim: Calculate the variance of monthly sales.

CODE CELL

```
sales = [12, 15, 14, 16, 20, 18, 25, 22, 19, 17, 13, 15]
mean = sum(sales) / len(sales)
variance = sum((x - mean) ** 2 for x in sales) / len(sales)
print(variance)
```

Output:

13.472222222222223

Aim: Compute the standard deviation of temperatures.

CODE CELL

```
import math

temps = [30, 32, 31, 30, 35, 33, 34]
mean = sum(temps) / len(temps)
variance = sum((x - mean) ** 2 for x in temps) / len(temps)
std_dev = math.sqrt(variance)
print(std_dev)
```

Output:

```
1.8070158058105026
```

Week 2 (Pre-processing Techniques)

Aim: A company collects data about customer behavior, including age, income, marital status, and purchase frequency. They want to focus only on income and purchase frequency for a targeted campaign. Select specific attributes from the dataset.

CODE CELL

```
import pandas as pd

data = {
    "Age": [25, 30, 35, 40],
    "Income": [40000, 50000, 60000, 70000],
    "Marital Status": ["Single", "Married", "Married", "Single"],
    "Purchase_Frequency": [5, 7, 6, 8]
}

df = pd.DataFrame(data)
selected = df[["Income", "Purchase_Frequency"]]
print(selected)
```

Output:

	Income	Purchase_Frequency
0	40000	5
1	50000	7
2	60000	6
3	70000	8

Aim: A dataset of student grades has some missing values. Fill these missing values using the mean of the column. Handle missing values in the dataset.

CODE CELL

```
import pandas as pd
import numpy as np

data = {
    "Student": ["Alice", "Bob", "Charlie", "David"],
    "Maths": [85, np.nan, 78, 90],
    "Science": [88, 92, np.nan, 85]
}

df = pd.DataFrame(data)
df["Maths"].fillna(df["Maths"].mean(), inplace=True)
df["Science"].fillna(df["Science"].mean(), inplace=True)
print(df)
```

Output:

	Student	Maths	Science
0	Alice	85.000000	88.000000
1	Bob	84.333333	92.000000
2	Charlie	78.000000	88.333333
3	David	90.000000	85.000000

Aim: A hospital has patient age data. Group patients into categories: "Young", "Middle-aged", and "Old". Discretize continuous data into categories.

CODE CELL

```
import pandas as pd

data = {"Age": [22, 35, 44, 50, 62, 70]}
df = pd.DataFrame(data)

bins = [0, 30, 50, 100]
labels = ["Young", "Middle-aged", "Old"]
df["Age_Group"] = pd.cut(df["Age"], bins=bins, labels=labels, right=False)
print(df)
```

Output:

	Age	Age_Group
0	22	Young
1	35	Middle-aged
2	44	Middle-aged
3	50	Old
4	62	Old
5	70	Old

Aim: Detect outliers in a dataset of product prices using the IQR method. Remove outliers from the dataset.

CODE CELL

```
import pandas as pd

prices = {"Price": [100, 200, 300, 400, 10000, 250, 275, 310]}
df = pd.DataFrame(prices)

Q1 = df["Price"].quantile(0.25)
Q3 = df["Price"].quantile(0.75)
IQR = Q3 - Q1

filtered_df = df[(df["Price"] >= Q1 - 1.5 * IQR) & (df["Price"] <= Q3 + 1.5 * IQR)]
print(filtered_df)
```

Output:

	Price
0	100
1	200
2	300
3	400
5	250
6	275
7	310

Week 3 (KNN algorithm)

Aim: A company has a dataset of customers labeled as "Likely to Purchase" or "Unlikely to Purchase" based on their age and income. Predict the label for a new customer. Implement KNN for classification.

CODE CELL

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Data
data = {
    "Age": [22, 25, 47, 52, 46, 56, 48, 40, 30, 28],
    "Income": [15000, 18000, 44000, 52000, 41000, 60000, 45000, 42000, 20000, 25000],
    "Purchase": ["No", "No", "Yes", "Yes", "Yes", "Yes", "Yes", "Yes", "No", "No"]
}
df = pd.DataFrame(data)

# Features and Labels
X = df[["Age", "Income"]]
y = df["Purchase"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# KNN Classifier
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)
print("Predicted Labels:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output:

```
Predicted Labels: ['Yes' 'No']
Accuracy: 1.0
```

Aim: A company tracks the monthly expenditures of households and wants to predict expenditure based on family size and income. Implement KNN for regression.

CODE CELL

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error

# Data
data = {
    "Family Size": [1, 2, 3, 4, 5, 6, 7, 8],
    "Income": [20000, 30000, 40000, 50000, 60000, 70000, 80000, 90000],
    "Expenditure": [1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500]
}
df = pd.DataFrame(data)

# Features and Labels
X = df[["Family Size", "Income"]]
y = df["Expenditure"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

# KNN Regressor
model = KNeighborsRegressor(n_neighbors=2)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
print("Predicted Expenditure:", y_pred)
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
```

Output:

```
Predicted Expenditure: [1500. 3500.]
Mean Squared Error: 0.0
```

Week 4 (Decision Tree Algorithm)

Aim: A bank has a dataset with customer information, including age, income, and whether or not they defaulted on a loan. The task is to predict whether a new customer is likely to default. Implement a decision tree classifier for this problem.

CODE CELL

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Data
data = {
    "Age": [25, 30, 35, 40, 45, 50, 55, 60, 65, 70],
    "Income": [20000, 30000, 40000, 50000, 60000, 70000, 80000, 90000, 100000, 110000],
    "Defaulted": ["No", "No", "No", "Yes", "Yes", "Yes", "Yes", "No", "No", "Yes"]
}
df = pd.DataFrame(data)

# Features and Labels
X = df[["Age", "Income"]]
y = df["Defaulted"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Decision Tree Classifier
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)
print("Predicted Labels:", y_pred)
print("Accuracy:", accuracy_score(y_test, y_pred))

# Decision Tree Structure
from sklearn.tree import export_text
print("Decision Tree Structure:")
print(export_text(model, feature_names=["Age", "Income"]))
```

Output:

```
Predicted Labels: ['No' 'No']
Accuracy: 1.0
Decision Tree Structure:
|--- Age <= 37.50
|   |--- class: No
|--- Age > 37.50
|   |--- Income <= 85000.00
|   |   |--- class: Yes
|   |--- Income > 85000.00
|   |   |--- Income <= 100000.00
|   |   |   |--- class: No
|   |   |   |--- Income > 100000.00
|   |   |   |   |--- class: Yes
```

Aim: Using the same customer loan default dataset, the goal is to optimize the decision tree by tuning its hyperparameters (e.g., max_depth, min_samples_split). Tune the decision tree for better classification performance.

CODE CELL

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier

# Data
data = {
    "Age": [25, 30, 35, 40, 45, 50, 55, 60, 65, 70],
    "Income": [20000, 30000, 40000, 50000, 60000, 70000, 80000, 90000, 100000, 110000],
    "Defaulted": ["No", "No", "No", "Yes", "Yes", "Yes", "Yes", "No", "No", "Yes"]
}
df = pd.DataFrame(data)

# Features and Labels
X = df[["Age", "Income"]]
y = df["Defaulted"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Grid Search for Hyperparameter Tuning
param_grid = {
    "max_depth": [3, 5, 10, None],
    "min_samples_split": [2, 5, 10],
}
model = DecisionTreeClassifier(random_state=42)
grid_search = GridSearchCV(model, param_grid, cv=5)
grid_search.fit(X_train, y_train)

# Best Hyperparameters
print("Best Hyperparameters:", grid_search.best_params_)

# Best Model
best_model = grid_search.best_estimator_

# Prediction
y_pred = best_model.predict(X_test)
print("Predicted Labels:", y_pred)
print("Best Accuracy:", grid_search.best_score_)

# Accuracy with Tuned Hyperparameters
from sklearn.metrics import accuracy_score
print("Accuracy with Tuned Hyperparameters:", accuracy_score(y_test, y_pred))
```

Output:

```
Best Hyperparameters: {'max_depth': 3, 'min_samples_split': 10}
Predicted Labels: ['Yes' 'Yes']
Best Accuracy: 0.7
Accuracy with Tuned Hyperparameters: 0.0
```

Aim: A real estate company wants to predict house prices based on features such as size (in square feet) and location. Implement a decision tree regressor to predict house prices.

CODE CELL

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

# Data
data = {
    "Size": [1500, 1800, 2000, 2500, 3000, 3500, 4000],
    "Price": [400000, 450000, 500000, 600000, 650000, 700000, 800000]
}
df = pd.DataFrame(data)

# Features and Labels
X = df[["Size"]]
y = df["Price"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Decision Tree Regressor
model = DecisionTreeRegressor(random_state=42)
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)
print("Predicted House Prices:", y_pred)

# Mean Squared Error
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
```

Output:

```
Predicted House Prices: [500000. 500000.]
Mean Squared Error: 625000000.0
```

Week 5 (Decision Tree Regressor for Salary Prediction)

Aim: You are given a dataset that contains the number of years of experience and the corresponding salaries of employees in a company. Using the Decision Tree Regressor, predict the salary of an employee based on their years of experience.

CODE CELL

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

# Load and Prepare the Data
# Example dataset with years of experience and corresponding salary
data = {
    "Years of Experience": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Salary": [40000, 42000, 46000, 49000, 54000, 60000, 65000, 70000, 75000, 80000]
}

df = pd.DataFrame(data)

# Features and Labels
X = df[["Years of Experience"]]
y = df["Salary"]

# Split the Data into Training and Test Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize the Decision Tree Regressor
model = DecisionTreeRegressor(random_state=42)
model.fit(X_train, y_train)

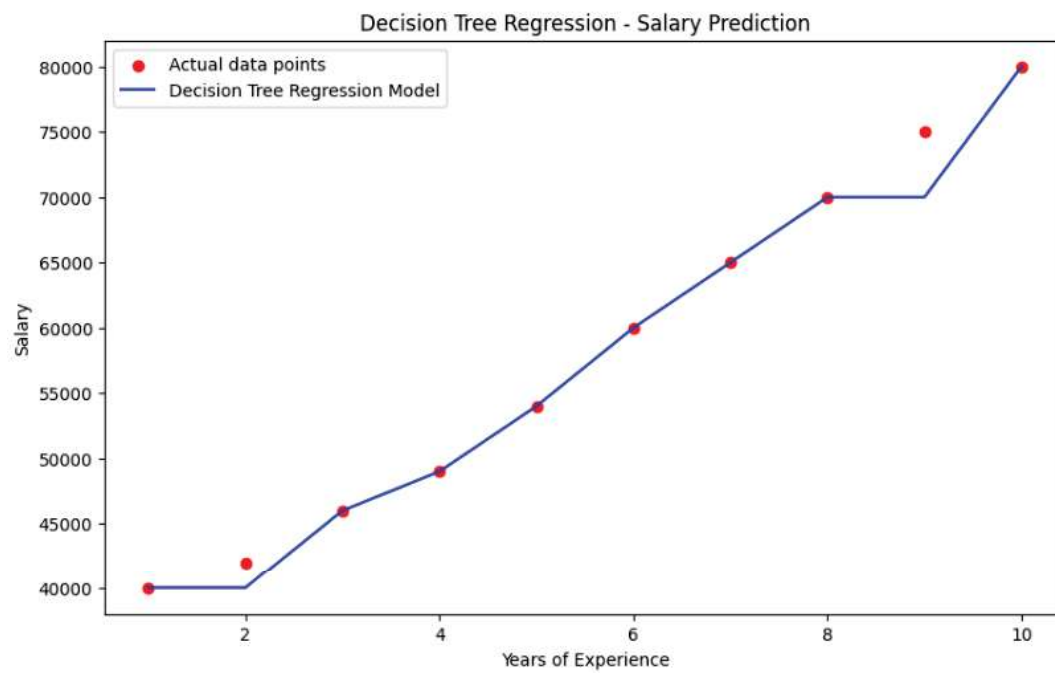
# Make Predictions
y_pred = model.predict(X_test)

# Evaluate the Model's Performance
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)

# Visualize the Decision Tree Regression Model
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='red', label='Actual data points')
plt.plot(X, model.predict(X), color='blue', label='Decision Tree Regression Model')
plt.title('Decision Tree Regression - Salary Prediction')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend()
plt.show()
```

Output:

Mean Squared Error: 14500000.0



Week 6 (Random Forest Regressor for Salary Prediction)

Aim: You are given a dataset that contains the number of years of experience and the corresponding salaries of employees in a company. Using the Random Forest algorithm, predict the salary of an employee based on their years of experience.

CODE CELL

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

# Prepare the data
# Example dataset with years of experience and corresponding salary
data = {
    "Years of Experience": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Salary": [40000, 42000, 46000, 49000, 54000, 60000, 65000, 70000, 75000, 80000]
}

df = pd.DataFrame(data)

# Features and Labels
X = df[["Years of Experience"]]
y = df["Salary"]

# Split the Data into Training and Test Sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train the model using Random Forest Regressor
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Make Predictions
y_pred = model.predict(X_test)

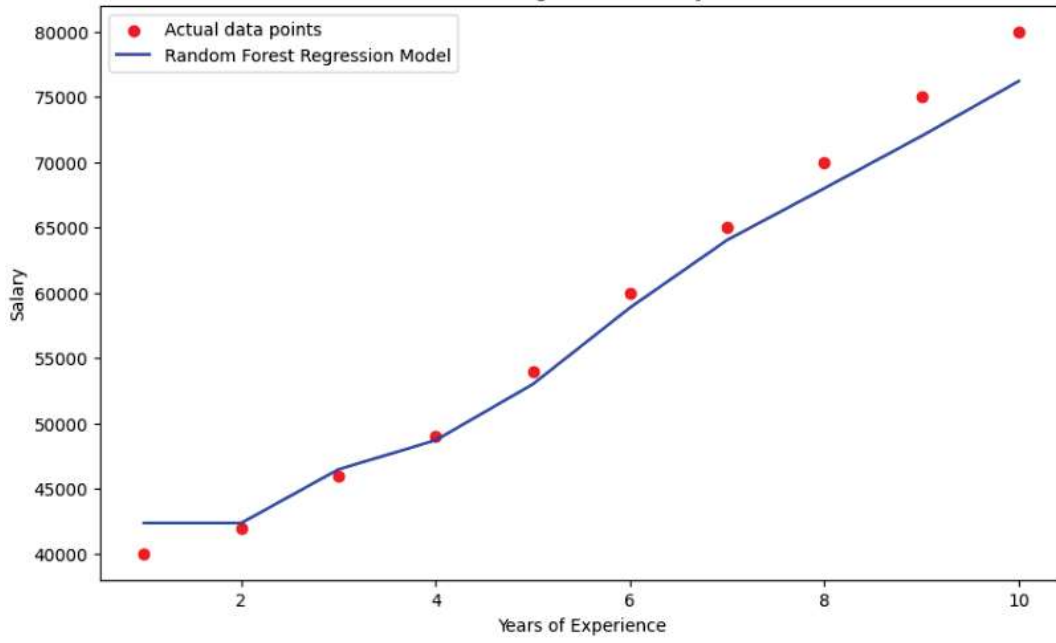
# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)

# Model Interpretation: Plotting the Decision Boundaries
plt.figure(figsize=(10, 6))
plt.scatter(X, y, color='red', label='Actual data points')
plt.plot(X, model.predict(X), color='blue', label='Random Forest Regression Model')
plt.title('Random Forest Regression - Salary Prediction')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend()
plt.show()
```

Output:

Mean Squared Error: 4568450.0

Random Forest Regression - Salary Prediction



Week 7 (Random Forest for Classification and Regression)

Aim: Given a dataset, apply the Random Forest algorithm to solve both the classification and regression problems.

CODE CELL

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import classification_report, mean_squared_error

# Example classification dataset (e.g., predicting if a customer will buy a product)
classification_data = {
    'Age': [22, 25, 47, 52, 46, 56, 48, 40, 30, 28],
    'Income': [15000, 18000, 44000, 52000, 41000, 60000, 45000, 42000, 20000, 25000],
    'Purchase': ['No', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No', 'No']
}

classification_df = pd.DataFrame(classification_data)

# Prepare classification features and target
X_class = classification_df[['Age', 'Income']]
y_class = classification_df['Purchase'].map({'Yes': 1, 'No': 0}) # Convert labels to 0 and 1

# Split the data into training and testing sets for classification
X_train_class, X_test_class, y_train_class, y_test_class = train_test_split(X_class, y_class, test_size=0.2, random_state=42)

# Train the Random Forest Classifier
classifier = RandomForestClassifier(n_estimators=100, random_state=42)
classifier.fit(X_train_class, y_train_class)

# Make predictions for classification
y_pred_class = classifier.predict(X_test_class)

# Evaluate the classification model
print("Classification Report:")
print(classification_report(y_test_class, y_pred_class))

# Visualize feature importances for the classifier
plt.figure(figsize=(10, 6))
plt.bar(X_class.columns, classifier.feature_importances_)
plt.title('Feature Importances for Classification Problem')
plt.xlabel('Features')
plt.ylabel('Importance')
plt.show()

# Example regression dataset (e.g., predicting house prices based on size and location)
regression_data = {
    'Size': [1500, 1800, 2000, 2500, 3000, 3500, 4000],
    'Price': [400000, 450000, 500000, 600000, 650000, 700000, 800000]
}

regression_df = pd.DataFrame(regression_data)

# Prepare regression features and target
X_reg = regression_df[['Size']]
y_reg = regression_df['Price']

# Split the data into training and testing sets for regression
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=0.2, random_state=42)

# Train the Random Forest Regressor
regressor = RandomForestRegressor(n_estimators=100, random_state=42)
regressor.fit(X_train_reg, y_train_reg)

# Make predictions for regression
y_pred_reg = regressor.predict(X_test_reg)
```

```
# Evaluate the regression model
mse = mean_squared_error(y_test_reg, y_pred_reg)
print("Mean Squared Error (Regression):", mse)

# Visualize the regression predictions
plt.figure(figsize=(10, 6))
plt.scatter(X_reg, y_reg, color='red', label='Actual data points')
plt.plot(X_reg, regressor.predict(X_reg), color='blue', label='Random Forest Regression Model')
plt.title('Random Forest Regression - Price Prediction')
plt.xlabel('Size (sq. ft.)')
plt.ylabel('Price')
plt.legend()
plt.show()

# Model Interpretation for Regression
plt.figure(figsize=(10, 6))
plt.bar(X_reg.columns, regressor.feature_importances_)
plt.title('Feature Importances for Regression Problem')
plt.xlabel('Features')
plt.ylabel('Importance')
plt.show()
```

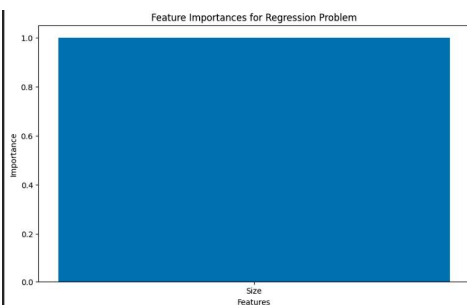
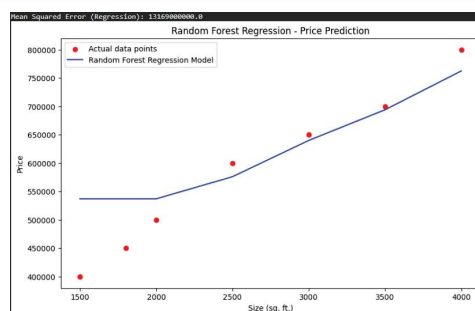
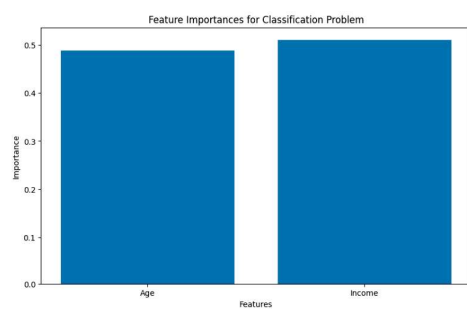
Output:

```
Classification Report:
              precision    recall  f1-score   support

     0               1.00      1.00      1.00         2

 accuracy               1.00
 macro avg              1.00      1.00      1.00         2
weighted avg              1.00      1.00      1.00         2

Mean Squared Error (Regression): 13169000000.0
```



Week 8 (Support Vector Machine - SVM)

Aim: You have a dataset that contains information about customers, such as their age, income, and other details. Use the Support Vector Machine (SVM) to solve the problem.

CODE CELL

```

# Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt

# Example dataset (Customer data with Age, Income, and Purchase decision)
data = {
    'Age': [22, 25, 47, 52, 46, 56, 48, 40, 30, 28],
    'Income': [15000, 18000, 44000, 52000, 41000, 60000, 45000, 42000, 20000, 25000],
    'Purchased': ['No', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No', 'No']
}

# Convert dataset into a pandas DataFrame
df = pd.DataFrame(data)

# Map 'Yes' and 'No' values to 1 and 0 for classification task
df['Purchased'] = df['Purchased'].map({'Yes': 1, 'No': 0})

# Prepare features and target variable
X = df[['Age', 'Income']]
y = df['Purchased']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Feature Scaling (Standardization of the features)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize the Support Vector Machine (SVM) Classifier
svm_classifier = SVC(kernel='linear', random_state=42)

# Train the model
svm_classifier.fit(X_train_scaled, y_train)

# Make predictions on the test set
y_pred = svm_classifier.predict(X_test_scaled)

# Evaluate the model
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Visualize the decision boundary (SVM with 2 features)
plt.figure(figsize=(10, 6))
plt.scatter(X_train['Age'], X_train['Income'], c=y_train, cmap='coolwarm', label='Training Data')
plt.scatter(X_test['Age'], X_test['Income'], c=y_test, marker='x', label='Test Data', edgecolor='black')

# Create meshgrid to plot decision boundary
xx, yy = np.meshgrid(np.linspace(X['Age'].min(), X['Age'].max(), 100),
                     np.linspace(X['Income'].min(), X['Income'].max(), 100))

# Predict on meshgrid
Z = svm_classifier.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot decision boundary
plt.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
plt.title('SVM Decision Boundary')
plt.xlabel('Age')
plt.ylabel('Income')
plt.legend()
plt.show()

# Model Interpretation
# Coefficients of the SVM decision boundary
print(f"Decision Boundary Coefficients: {svm_classifier.coef_}")
print(f"Intercept: {svm_classifier.intercept_}")

```

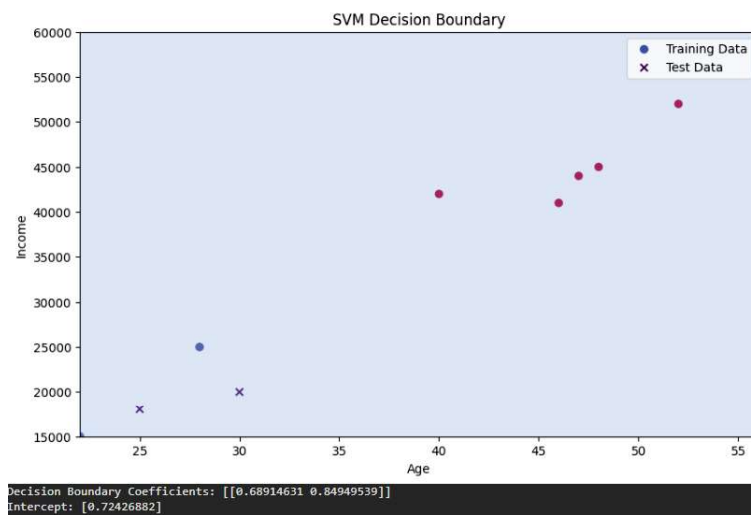
Output:

```
Classification Report:
      precision    recall  f1-score   support

     0       1.00      1.00      1.00         2
     1       1.00      1.00      1.00         1

 accuracy          1.00          1.00          1.00         3
 macro avg       1.00      1.00      1.00         3
 weighted avg    1.00      1.00      1.00         3

Decision Boundary Coefficients: [[0.68914631 0.84949539]]
Intercept: [0.72426882]
```



Week 9 (Simple Linear Regression)

Aim: Demonstrate simple linear regression algorithm for a regression problem.

CODE CELL

```
# Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Example dataset (Years of experience vs Salary)
data = {
    'Years Experience': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Salary': [40000, 45000, 50000, 55000, 60000, 65000, 70000, 75000, 80000, 85000]
}

# Convert dataset into a pandas DataFrame
df = pd.DataFrame(data)

# Prepare features and target variable
X = df[['Years Experience']] # Feature: Years of Experience
y = df['Salary'] # Target: Salary

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Initialize the Linear Regression model
linear_regressor = LinearRegression()

# Train the model on the training data
linear_regressor.fit(X_train, y_train)

# Make predictions on the test data
y_pred = linear_regressor.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred) # Mean Squared Error
r2 = r2_score(y_test, y_pred) # R-squared score

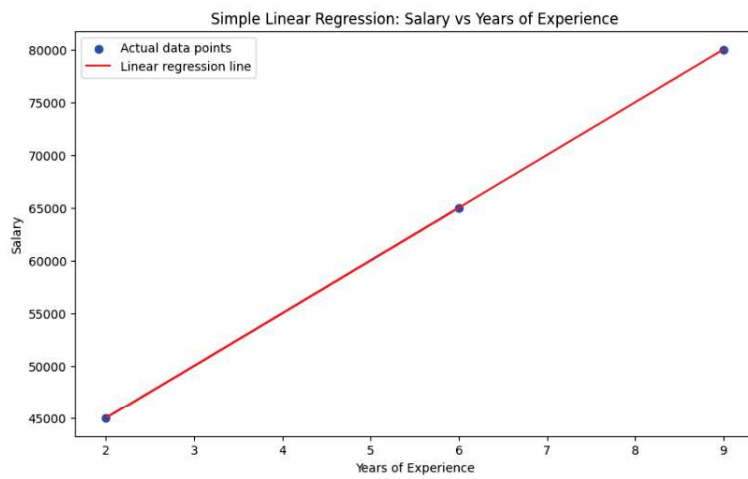
print(f"Mean Squared Error (MSE): {mse}")
print(f"R-squared Score: {r2}")

# Plot the results
plt.figure(figsize=(10, 6))
plt.scatter(X_test, y_test, color='blue', label='Actual data points')
plt.plot(X_test, y_pred, color='red', label='Linear regression line')
plt.title('Simple Linear Regression: Salary vs Years of Experience')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend()
plt.show()

# Model Interpretation
# Coefficients and intercept
print(f"Coefficient (slope): {linear_regressor.coef_}")
print(f"Intercept: {linear_regressor.intercept_}")
```


Output:

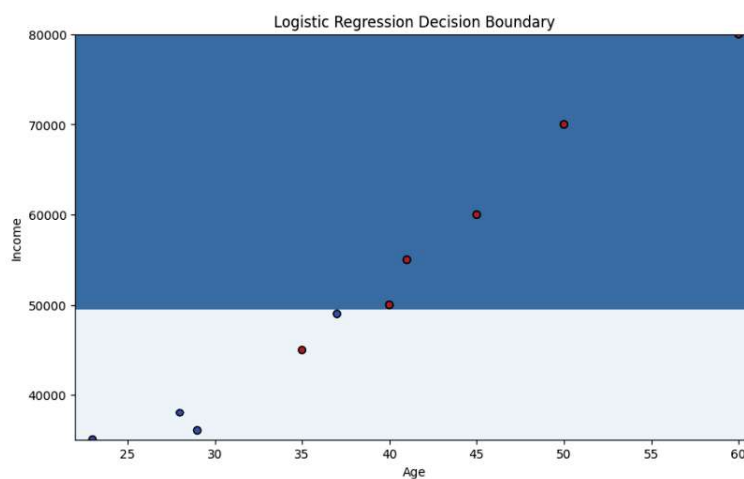
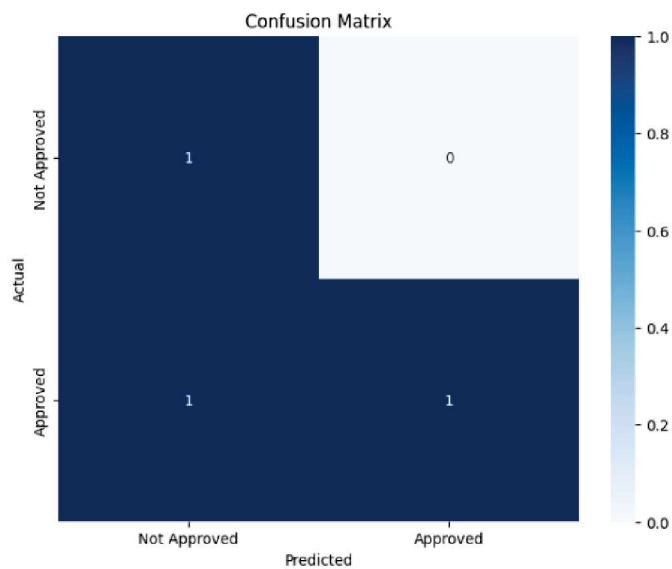
```
Mean Squared Error (MSE): 1.764651973446459e-23  
R-squared Score: 1.0  
Coefficient (slope): [5000.]  
Intercept: 35000.0
```



Week 10 (Logistic Regression)

Aim: You have been provided with a dataset containing the following features of customers of a bank: age, income, and loan approval status (1 for approved, 0 for not approved). You are required to build a Logistic Regression model to predict whether a customer will be approved for a loan based on their age and income.

CODE CELL



```

# Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns

# Example dataset (Age, Income, Loan Approval Status)
data = {
    'Age': [23, 35, 45, 50, 28, 60, 40, 37, 29, 41],
    'Income': [35000, 45000, 60000, 70000, 38000, 80000, 50000, 49000, 36000, 55000],
    'Loan_Approved': [0, 1, 1, 1, 0, 1, 1, 0, 0, 1]
}

# Convert dataset into pandas DataFrame
df = pd.DataFrame(data)

# Prepare features (X) and target variable (y)
X = df[['Age', 'Income']] # Features: Age, Income
y = df['Loan_Approved'] # Target: Loan Approval Status

# Split the data into training and testing sets (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Initialize the Logistic Regression model
log_reg_model = LogisticRegression()

# Train the Logistic Regression model on the training data
log_reg_model.fit(X_train, y_train)

# Make predictions on the test data
y_pred = log_reg_model.predict(X_test)

# Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred) # Accuracy of the model
conf_matrix = confusion_matrix(y_test, y_pred) # Confusion Matrix
class_report = classification_report(y_test, y_pred) # Classification Report

print(f"Accuracy: {accuracy}")
print(f"Confusion Matrix:\n{conf_matrix}")
print(f"Classification Report:\n{class_report}")

# Visualize the confusion matrix using a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Approved', 'Approved'],
            yticklabels=['Not Approved', 'Approved'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Model Interpretation
# Get the coefficients of the model
print(f"Model Coefficients (Age and Income): {log_reg_model.coef_}")
print(f"Model Intercept: {log_reg_model.intercept_}")

# Visualize the decision boundary
# Create a grid to plot decision boundary
x_min, x_max = X['Age'].min() - 1, X['Age'].max() + 1
y_min, y_max = X['Income'].min() - 1, X['Income'].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1), np.arange(y_min, y_max, 0.1))

# Predict on the grid and plot
Z = log_reg_model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

plt.figure(figsize=(10, 6))
plt.contourf(xx, yy, Z, alpha=0.75, cmap='Blues')
plt.scatter(X['Age'], X['Income'], c=y, edgecolors='k', marker='o', cmap='coolwarm')
plt.xlabel('Age')
plt.ylabel('Income')
plt.title('Logistic Regression Decision Boundary')
plt.show()

```

Output:

Accuracy: 0.6666666666666666

Confusion Matrix:

```
[[1 0]
 [1 1]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.50	1.00	0.67	1
1	1.00	0.50	0.67	2
accuracy			0.67	3
macro avg	0.75	0.75	0.67	3
weighted avg	0.83	0.67	0.67	3

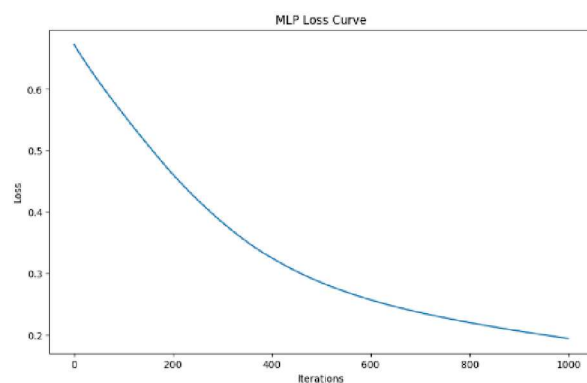
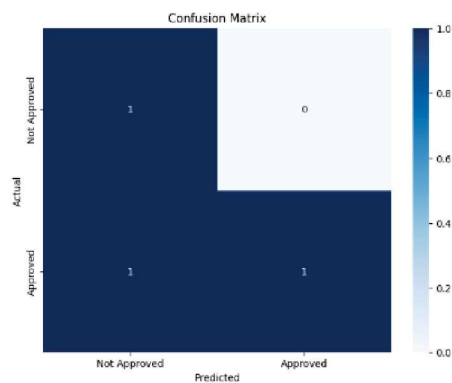
Model Coefficients (Age and Income): [[6.44700986e-05 2.14952720e-02]]

Model Intercept: [-1064.01847865]

Week 11 (Multi-layer Perceptron)

Aim: You are tasked with building a Multi-layer Perceptron (MLP) model to classify a set of customers into two categories based on their age, income, and loan approval status. The dataset provided has three features: age, income, and loan approval status. You are required to predict the loan approval status using the MLP algorithm.

CODE CELL



```

# Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns

# Example dataset (Age, Income, Loan Approval Status)
data = {
    'Age': [23, 35, 45, 50, 28, 60, 40, 37, 29, 41],
    'Income': [35000, 45000, 60000, 70000, 38000, 80000, 50000, 49000, 36000, 55000],
    'Loan_Approved': [0, 1, 1, 1, 0, 1, 1, 0, 0, 1]
}

# Convert dataset into pandas DataFrame
df = pd.DataFrame(data)

# Prepare features (X) and target variable (y)
X = df[['Age', 'Income']] # Features: Age, Income
y = df['Loan_Approved'] # Target: Loan Approval Status

# Split the data into training and testing sets (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Data Preprocessing: Feature Scaling (Standardization)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize and train the Multi-layer Perceptron Classifier
mlp_model = MLPClassifier(hidden_layer_sizes=(5,), max_iter=1000, random_state=42)
mlp_model.fit(X_train_scaled, y_train)

# Make predictions on the test set
y_pred = mlp_model.predict(X_test_scaled)

# Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred) # Accuracy of the model
conf_matrix = confusion_matrix(y_test, y_pred) # Confusion Matrix
class_report = classification_report(y_test, y_pred) # Classification Report

print(f"Accuracy: {accuracy}")
print(f"Confusion Matrix:\n{conf_matrix}")
print(f"Classification Report:\n{class_report}")

# Visualize the confusion matrix using a heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Approved', 'Approved'],
            yticklabels=['Not Approved', 'Approved'])
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Model Interpretation
# Visualize the weights of the model (Coefficients of the hidden layer)
print(f"Model Weights (Hidden Layer): {mlp_model.coefs_}")

# Visualize the model's learning curve
plt.figure(figsize=(10, 6))
plt.plot(mlp_model.loss_curve_)
plt.title('MLP Loss Curve')
plt.xlabel('Iterations')
plt.ylabel('Loss')
plt.show()

```

Output:

Accuracy: 0.6666666666666666

Confusion Matrix:

```
[[1 0]
 [1 1]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.50	1.00	0.67	1
1	1.00	0.50	0.67	2
accuracy			0.67	3
macro avg	0.75	0.75	0.67	3
weighted avg	0.83	0.67	0.67	3

Model Weights (Hidden Layer): [array([[-0.7343864 , 0.17855095, 1.46569854, 0.16172467, -0.05176324],
[-1.14526746, -1.40771544, 1.63828581, 0.16647491, 0.00184087]]), array([[-1.29960988],
[-1.04606506],
[0.72797868],
[-0.11225933],
[-0.00374214]])]

Week 12 (K-means Clustering)

Aim: Implement the K-means algorithm and apply it to the selected data. Evaluate performance by measuring the sum of the Euclidean distance of each example from its class center. Test the performance of the algorithm as a function of the parameter K .

CODE CELL


```

# Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
from sklearn.metrics import pairwise_distances_argmin_min

# Example dataset: Simulating customer data with age and income
data = {
    'Age': [23, 35, 45, 50, 28, 60, 40, 37, 29, 41, 55, 32, 33, 47, 44],
    'Income': [35000, 45000, 60000, 70000, 38000, 80000, 50000, 49000, 36000, 55000, 65000, 45000, 41000, 52000, 61000]
}

# Convert dataset into pandas DataFrame
df = pd.DataFrame(data)

# Prepare features (X) for clustering
X = df[['Age', 'Income']]

# Data Preprocessing: Feature Scaling (Standardization)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Function to apply K-means and calculate the performance
def apply_kmeans(X_scaled, k):
    # Initialize KMeans with k clusters
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)

    # Sum of Euclidean distances (inertia) for each data point from its cluster center
    inertia = kmeans.inertia_

    return kmeans, inertia

# Evaluate performance with different values of K
K_values = range(1, 11)
inertias = []

for k in K_values:
    _, inertia = apply_kmeans(X_scaled, k)
    inertias.append(inertia)

# Plot Inertia for each value of K (Elbow Method)
plt.plot(K_values, inertias, marker='o')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia (Sum of Euclidean Distances)')
plt.show()

# Based on the elbow method, let's choose an optimal K (e.g., K=3)
optimal_k = 3
kmeans_model, _ = apply_kmeans(X_scaled, optimal_k)

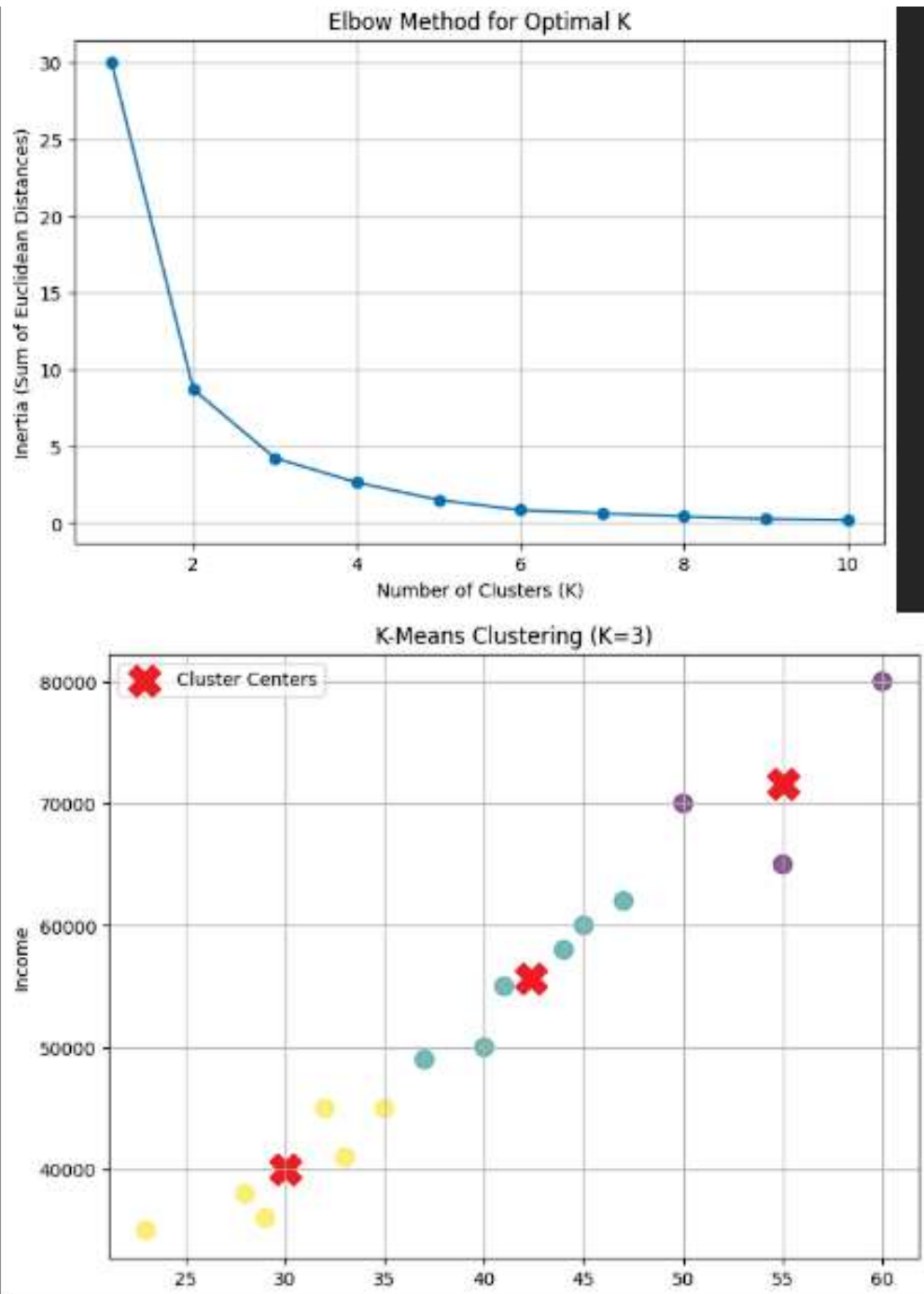
# Visualize the clusters and their centers
df['Cluster'] = kmeans_model.labels_
plt.scatter(df['Age'], df['Income'], c=df['Cluster'], cmap='viridis', marker='o')
plt.scatter(kmeans_model.cluster_centers_[:, 0], kmeans_model.cluster_centers_[:, 1],
            s=200, c='red', marker='X', label='Cluster Centers')
plt.title(f'K-means Clustering (K={optimal_k})')
plt.xlabel('Age')
plt.ylabel('Income')
plt.legend()
plt.show()

# Final performance evaluation
# Calculate the sum of the Euclidean distances of each point from its center
distances = pairwise_distances_argmin_min(X_scaled, kmeans_model.cluster_centers_)[1]
sum_of_distances = np.sum(distances)
print(f"Sum of Euclidean distances from each point to its cluster center: {sum_of_distances}")

```

Output:

Sum of Euclidean distances from each point to its cluster center: 8.006639187239042



Week 13 (Fuzzy C-Means Clustering)

Aim: Demonstrate the use of the Fuzzy C-Means (FCM) clustering algorithm on a dataset of customer transactions. The dataset contains features like age, income, and spending score. Cluster the customers into groups and evaluate the results.

CODE CELL

```
# Import necessary libraries
import numpy as np
import pandas as pd
import skfuzzy as fuzz
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

# Step 1: Prepare the Dataset
data = {
    'Age': [25, 32, 47, 51, 23, 37, 45, 29, 34, 50, 60, 22, 41, 35, 43],
    'Income': [35000, 40000, 60000, 70000, 32000, 45000, 55000, 38000, 43000, 69000, 80000, 31000, 49000, 52000, 61000],
    'Spending_Score': [40, 60, 30, 70, 50, 65, 35, 55, 58, 75, 80, 45, 68, 59, 72]
}

# Convert the dataset into a pandas DataFrame
df = pd.DataFrame(data)

# Step 3: Data Preprocessing - Standardize the data
scaler = StandardScaler()
data_scaled = scaler.fit_transform(df)

# Step 4: Apply Fuzzy C-Means Algorithm
# Set the number of clusters
n_clusters = 3
cntr, u, u0, d, jm, p, fpc = fuzz.cluster.cmeans(data_scaled.T, n_clusters, 2, error=0.005, maxiter=1000)

# The matrix 'u' represents the membership matrix
# Each element in u indicates the degree of membership of a point in a cluster (between 0 and 1)

# Step 5: Evaluate the Performance
# Fuzzy Partition Coefficient (FPC) - Indicates the fuzziness of the clustering
print(f"Fuzzy Partition Coefficient: {fpc}")

# Visualize the clusters
plt.figure(figsize=(10, 6))
for i in range(n_clusters):
    plt.scatter(df['Age'], df['Income'], c=u[i, :], cmap='viridis', label=f'Cluster {i+1}')

plt.title('Fuzzy C-Means Clustering of Customer Data')
plt.xlabel('Age')
plt.ylabel('Income')
plt.legend()
plt.show()

# Step 6: Interpret the Results
# The membership matrix 'u' indicates the degree of membership of each data point to each cluster.
# The closer the value is to 1, the stronger the membership to that particular cluster.
print("\nMembership Matrix (first 5 rows):")
print(u[:, :5])
```

Output:

Fuzzy Partition Coefficient: 0.7364666653904776

Membership Matrix (first 5 rows):

```
[[0.03982728 0.76403711 0.41690315 0.01367079 0.05762459]
 [0.00825826 0.0319309 0.26680525 0.98156778 0.01023877]
 [0.95191445 0.20403198 0.3162916 0.00476144 0.93213663]]
```



Week 14 (Expectation Maximization Clustering)

Aim: Demonstrate the use of the Expectation Maximization (EM) clustering algorithm on a dataset of customer transactions. The dataset contains features like age, income, and spending score. Apply the EM algorithm to cluster the customers into groups and evaluate the results.

CODE CELL

```
# Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.mixture import GaussianMixture
import matplotlib.pyplot as plt

# Step 1: Prepare the Dataset
# Simulate customer data
data = {
    'Age': [25, 32, 47, 51, 23, 37, 45, 29, 34, 50, 60, 22, 41, 35, 43],
    'Income': [35000, 40000, 60000, 70000, 32000, 45000, 55000, 38000, 43000, 69000, 80000, 31000, 49000, 52000, 61000],
    'Spending_Score': [40, 60, 30, 70, 50, 65, 35, 55, 58, 75, 80, 45, 68, 59, 72]
}

# Convert the dataset into a pandas DataFrame
df = pd.DataFrame(data)

# Step 3: Data Preprocessing - Standardize the data
scaler = StandardScaler()
data_scaled = scaler.fit_transform(df)

# Step 4: Apply the Expectation Maximization (EM) Algorithm using Gaussian Mixture Model
# Set the number of clusters (e.g., 3)
n_clusters = 3
gmm = GaussianMixture(n_components=n_clusters, random_state=42)
gmm.fit(data_scaled)

# Get the predicted cluster labels
labels = gmm.predict(data_scaled)

# Step 5: Evaluate the Performance
# Print the log-likelihood, a measure of model fit
print(f"Log-likelihood: {gmm.score(data_scaled)}")

# Step 6: Interpret the Results
# Visualize the clusters
plt.figure(figsize=(10, 6))
plt.scatter(df['Age'], df['Income'], c=labels, cmap='viridis')
plt.title('Expectation Maximization Clustering of Customer Data')
plt.xlabel('Age')
plt.ylabel('Income')
plt.colorbar(label='Cluster ID')
plt.show()

# Analyzing the cluster centers
print("\nCluster centers (in original space):")
print(scaler.inverse_transform(gmm.means_))

# Print the covariance matrices
print("\nCovariance matrices:")
for i, cov in enumerate(gmm.covariances_):
    print(f"\nCluster {i+1}:")
    print(cov)
```

Output:

Log-likelihood: 0.7245694921097823

Cluster centers (in original space):

```
[[3.92376925e+01 4.84874502e+04 5.58727200e+01]
 [5.36666667e+01 7.30000000e+04 7.50000000e+01]
 [2.47461413e+01 3.39966272e+04 4.74880854e+01]]
```

Covariance matrices:

Cluster 1:

```
[[ 0.22932558  0.20986325 -0.25031226]
 [ 0.20986325  0.19835397 -0.28029877]
 [-0.25031226 -0.28029877  0.96997394]]
```

Cluster 2:

```
[[0.17304428 0.1448401  0.09587716]
 [0.1448401  0.12124357 0.08073876]
 [0.09587716 0.08073876 0.07957491]]
```

Cluster 3:

```
[[0.06183785 0.04727827 0.05201059]
 [0.04727827 0.03706835 0.03033229]
 [0.05201059 0.03033229 0.14920184]]
```

